# Sharpline Model Plan

Oct 21, 2026 · @George

## Data rule

Use nflverse for anything it provides, and screenshots only for what it lacks. nflverse is free, has history back to the 2010s, and defines every stat the same way each season.

- nflverse supplies schedules, scores, scoring plays, historical spreads, totals and moneylines, weather, injuries, player usage (targets, carries, red zone looks, snap counts), EPA, PROE, and, per its participation data, personnel, pressure, man or zone, box counts, pass rushers and time to throw.
- Screenshots supply sportsbook prop lines and anytime-TD prices (no free history), yards per target allowed to slot and outside receivers, play clock stats, gear change, adjusted plays, and any Sharp stat that fails the overlap test below.
- The overlap test runs before any training. Rebuild 2026 through Week 5 from nflverse and compare 32 teams against your captures. A correlation above 0.9 and a small average gap means the nflverse version is used. The personnel table's 9,597 charted plays is the first checksum.
- If both sources pass, the model reads nflverse and the screenshots stay as an audit copy.

You download nflverse files once and place them in `data/extracted/` with a sidecar, like any other CSV. Claude Code never fetches anything. Participation data seasons are unconfirmed, so the test also tells us how far back each stat goes.

## Shared foundation

Every model is judged against the market's no-vig closing probability, not against 50%. A model that doesn't beat the market's log loss has no edge, whatever its hit rate says.

- Raw prices are stored. De-vigging is applied at read time, so the method can change later without recapturing anything. US retail moneylines start with the same-percentage method.
- Scores are log loss and calibration on games the model hasn't seen, compared with the market on the same games.

- Closing line value (did our number beat the closing price) is tracked from week one. It shows signal in weeks. A 55% spread record needs about 1,500 bets to separate from luck.
- Walk-forward only: a prediction uses data from before the game (`as_of_week` rule).
- One simulator sits behind every bet: team ratings give expected points, expected points give plays and pass rate, plays are divided among players, then 10,000 simulated games give every probability from the same runs.
- Method follows data size: hand-set weights first, then ridge or Bayesian regression at a few hundred rows, then gradient boosting at thousands.

## Spread

The model predicts how far each game's result lands from the posted spread, not the margin from scratch. NFL spreads are the sharpest prices in the sport, so the realistic goal is a small, honest edge and a trustworthy benchmark for the other bets.

Target: the home margin minus the market spread, shrunk toward zero. The cover probability is the chance that margin beats the spread, using a normal curve (NFL margins have a standard deviation near 13 points, to be fitted).

Inputs for v0:

- Offense EPA per play against the opposing defense's, and points per drive
- Pace (plays per game) and the PROE matchup
- Pass protection against pass rush (pressure allowed against pressure rate)
- Small adjustments for box counts, man or zone and positional yards per target
- Home field, rest days, and a manual flag for a starting QB out

Method ladder:

1. v0: hand-set weights on the differences, written down in code.
2. v1: ridge regression on the same differences. nflverse history from 2016 gives about 2,700 games, enough for this if the features pass the overlap test.
3. v2: Bayesian offense and defense ratings per team, updated weekly. Boosting only if it beats ridge on held-out seasons.

Edge test: model cover probability against the de-vigged price (about 50% at -110, breakeven 52.4% before de-vig). Bet only when the gap in points between model and line clears a minimum.

Grading: hit, miss, or push, plus closing line value and log loss against the market.

Decisions for you: the minimum point gap to bet (I'd start at 1.5), and whether v0 includes the QB-out flag (I'd say yes, since a QB change moves a spread by 3 to 7 points).

## Moneyline

The moneyline needs no separate model. The spread model's predicted margin and its spread of outcomes give a win probability, and that probability is compared with the de-vigged moneyline price.

- Win probability: the chance the simulated margin is above zero. Fit a logistic curve on the margin so favorites and underdogs are calibrated (a 20% prediction wins about 20% of the time).
- Market probability: de-vig the two prices with the same-percentage method for US books, and keep the raw prices so the method can change.
- Edge: model probability minus market probability. Underdogs at long prices need a larger gap, because small errors in tail probabilities move the price a lot.
- Grading: win or loss, with ties as pushes. Log loss and calibration by price bucket (underdogs priced 10 to 20%, 20 to 35%, and so on) tell us where the model leaks.

Decisions for you: the minimum probability gap (I'd start at 4 points) and a cap on how long a price we'll back (I'd start at +300, since tail calibration needs hundreds of games).

## Game total

The total is predicted separately from the margin, because points depend on pace and efficiency while the margin depends on the gap between teams. Folding it into the same simulator keeps both consistent: the simulator produces each team's score, so margin and total come from the same runs.

Inputs for v0:

- Plays per game and plays per drive for both teams (pace)
- Points per drive and explosive play rate for the offense, against points per drive allowed by the defense
- Neutral pass rate and no-huddle rate (a faster, pass-heavy game scores and stops the clock more)
- Weather (wind and temperature) and roof, available in nflverse history

- Red zone efficiency, once it is loaded

Method ladder:

1. v0: expected plays per team times expected points per play, with hand-set weights.
2. v1: regression on total points, or a Poisson-style model of scoring drives, trained on nflverse history.
3. v2: simulate drives and plays, which also feeds the player models.

Edge test: the probability the simulated total goes over the posted number, against the de-vigged over and under prices. Totals are close to even money, so the de-vig method barely matters here.

Grading: over, under, or push (a total landing on the number is a push, and on key numbers like 41 or 44 it happens often).

Decisions for you: whether totals are in v0 (I'd say yes), and whether weather is a v0 input or waits for v1 (I'd include wind above 15 mph from the start).

## Receiving yards and receptions

The model predicts a full distribution for each tracked player, so any posted line can be priced, not just one.

Build the stat in three parts, then simulate:

1. Targets: the team's expected pass attempts (from the game simulator) times the player's target share.
2. Catches: targets times catch rate.
3. Yards: catches times yards per catch, with the long tail that yardage has.

Distributions: targets and receptions follow a negative binomial (counts that vary more than a Poisson allows). Yards per catch is skewed, so use a gamma or lognormal. Simulate the three together so a big game shows up in all of them.

Handling thin data: a player with 4 games shouldn't be judged on 4 games. A hierarchical model pulls each player's target share toward similar players at his position and role, and moves toward his own numbers as games accumulate. v0 can approximate this with a weighted blend of the last four weeks and the player's season rate.

Opponent adjustment: yards per target allowed to the position, shrunk toward the league average. Slot versus outside needs the screenshot data, since nflverse has no alignment field.

Edge test: probability of the over from the simulated distribution, against the de-vigged over and under prices. Book prices on props carry more margin than sides, so the gap has to clear a larger minimum.

Grading: over, under, or push (integer lines push), tracked by prop type and by line size.

Decisions for you: how many players to track per week (I'd say 40, picked by target share), and whether v0 ignores injuries to teammates or applies a manual target-share bump.

## Rushing yards

Rushing props depend on game script more than any other prop. A team that leads runs more in the second half, and a team that trails abandons the run. The model has to take the script from the game simulator, not from a player's average.

Build the stat in two parts, then simulate:

1. Carries: the team's expected rush attempts (from the simulator) times the player's carry share, with a goal-line and short-yardage share tracked separately.
2. Yards: carries times yards per carry, where yards per carry is drawn from a skewed distribution (most runs are short, a few are long).

Inputs: opponent run defense (yards before contact allowed, rush stuff rate, box counts), the offensive line's yards before contact, the player's own yards after contact from recent games, and the spread (a 7-point favorite gets more carries late).

Distribution: carries as a negative binomial, yards per carry as a gamma or lognormal, simulated together. Treat a running QB as a separate case, since his carries come from different plays.

Edge test and grading: the same as receiving props, with pushes on integer lines.

What's weaker here: yards before contact comes from Sharp's screenshots or from Pro Football Reference, and the overlap test decides which. Without it, v0 leans on the opponent's yards per carry allowed from nflverse.

Decisions for you: whether v0 includes running QBs (I'd exclude them at first), and whether the model may bet a carries prop (I'd leave it out until yardage props are graded).

## Anytime TD

This is a probability problem, and the main risk is splitting a team's touchdowns badly among its players. The fix is to fix the pool first, then divide it.

1. Team touchdown pool: expected points for the team (from the game total and spread) turned into expected touchdowns, with field goals taken out.
2. Split by role: each player's share of the pool, based on red zone touches, goal-line carries, end zone targets, and the share of team touches inside the 10. Use a softmax or Dirichlet split so shares always sum to the pool.
3. Convert to a probability: P(at least one TD) = 1 - e^(-rate), where rate is the player's expected TDs.

Why this order: when one back takes every goal-line carry, his share of the pool should rise and his teammates' fall. Spreading a star's share across the roster by default would underprice him.

Inputs: red zone and goal-line usage over the last four to six games (nflverse has the play data), opponent touchdowns allowed by position, team implied points, and injuries to teammates who share the work.

Method ladder: v0 is a hand-weighted score of recent red zone share and team implied points. v1 trains a logistic or boosted model on nflverse player-games, which number in the tens of thousands over a decade, so boosting is reasonable here.

Edge test: model probability against the de-vigged anytime price. Anytime prices carry heavy margin, so a gap under about 4 points is noise.

Grading: scored or didn't, plus calibration by probability bucket. This bet has the most graded rows per week, so it validates fastest.

Decisions for you: whether to include QB rushing TDs (I'd include, since books price them), and whether to track defensive and special teams scorers (I'd leave them out).

## Edge, staking and the Value Board

An edge is the model's probability minus the market's de-vigged probability, after shrinking the model toward the market. A raw gap is almost always bigger than the true one, because a model is least reliable exactly where it disagrees with the market most.

- Shrinkage: blend model and market, for example 50/50 early in the season, moving toward the model as graded results show it beats the market's log loss.
- Minimum edge by bet type: spread 1.5 points, moneyline 4 points of probability, totals 1.5 points, props and anytime TD 4 to 5 points, since their margins are larger. These are starting values to revisit after 100 graded bets per type.

- Stake size: quarter Kelly at most, capped per bet and per game. Bets in the same game are correlated, so the cap applies to the game, not just the bet.
- Closing line value: for every pick, store the price at pick time and the closing price. It is the earliest honest signal, and it comes with no extra data work.
- The Value Board ranks edges across all bet types, labeled with the model version, the sample size behind that model, and the shrinkage used, so a prop edge from an untested model doesn't outrank a tested one.

## Open decisions

All seven decisions were settled on Oct 22. Each one changes the build plan as shown.

| Decision | Decided | What it changes (spec v4 steps) |
|---|---|---|
| Historical backfill from nflverse | Download 2018 on, train from 2021 on, recent seasons weighted more; test both windows | Steps 12, 13, 19, 28 |
| Overlap test before training | Yes, right after the Week 6 import | Step 14 |
| Totals in v0 | Yes, same simulator | Step 17 |
| Spread minimum gap | 1.5 points | Step 26 |
| Moneyline price cap | +300 until 300 graded games | Steps 26 and 27 |
| Players tracked per week | 20 | Steps 30 and 31 (about 8 screenshots a week) |
| QB-out flag in v0 | Yes, manual | Step 17 and the `games` table |

Build order after the overlap test: game model (spread, moneyline, total), then receiving props, then anytime TD, then rushing props. Anytime TD grades fastest, so its results arrive first even though it is built third.